

Name: Yujun Wang

ID: 20348195

Submission date: 2025-11-09

Stubbing vs Mocking

Stubbing and mocking are both methods in testing where a function call is replaced with another function/output. This allows for the function being tested to not interact with outside elements or state which may affect how the tests run but instead to use predetermined and predictable outputs. This makes tests involving these stubs and mocks repeatable and predictable.

Stubbing refers only to replacing the output of a called function. This is useful when it doesn't matter what the parameters of the function being called are. For example when calling `datetime.now()`, there are no parameters and only the output matters. In this case stubbing is suitable because there are no parameters.

Mocking is more complex, it involves checking and verifying the parameters being passed to the function. This allows checking the logic to see if the flow of the logic in the function is correct. It is about verifying that previous behavior was correctly run then passing an output to continue the execution. An example of this would be the `datetime.fromisoformat(string)`. It takes an iso format string and converts it to a `datetime` object. In this case what is important isn't the `datetime` object output but rather the string input. A mock of this function allows us to verify that the correct args were passed.

Test Execution

All commands are run from the root of the project(same folder as `main.js`).

```
# run all tests
> pytest tests/
# generate coverage results
> pytest --cov=services --cov-report=html --cov-report=term tests/
# view coverage results(linux)
> sensible-browser ./tests/htmlcov/index.html
# view coverage results(windows)
> start ./tests/htmlcov/index.html
```

Test Case Summary(for new tests)

This only covers `test_pay_fees.py` and `test_refund.py`. The other files were updated to use mocking and stubbing but will not be described here. If a function only checks if it is not run then it will not be listed in either stubs or mocks.

`test_pay_fees.py`

Function Name	Purpose	Stubs	Mocks
<code>test_patron_id_short</code>	Test with a short patron id(must be 6 digits)		
<code>test_patron_id_long</code>	Test with a long patron id(must be 6 digits)		
<code>test_patron_id_alphabetic</code>	Test with a alphabetic patron id(must be 6 digits)		
<code>test_no_fees</code>	Test with no fees owed	<code>calculate_late_fee_for_book</code>	
<code>test_negative_fees</code>	Test with negative fees owed	<code>calculate_late_fee_for_book</code>	
<code>test_missing_fees</code>	Test with missing fee data	<code>calculate_late_fee_for_book</code>	
<code>test_invalid_book</code>	Test with no book found	<code>calculate_late_fee_for_book</code>	<code>get_book_by_id</code>
<code>test_pay_fee_invalid</code>	Test with PaymentGateway process_payment unsuccessful	<code>calculate_late_fee_for_book</code> <code>PaymentGateway</code>	<code>get_book_by_id</code> <code>process_payment</code>
<code>test_pay_fee_success</code>	Test with everything valid	<code>calculate_late_fee_for_book</code> <code>PaymentGateway</code>	<code>get_book_by_id</code> <code>process_payment</code>
<code>test_pay_fee_exception</code>	Test with PaymentGateway process_payment throwing an Exception	<code>calculate_late_fee_for_book</code> <code>PaymentGateway</code>	<code>get_book_by_id</code> <code>process_payment</code>

`test_refund.py`

Function Name	Purpose	Stubs	Mocks
<code>test_invalid_transaction_id</code>	Test with a invalid transaction id(must start with "txn_")	<code>PaymentGateway</code>	
<code>test_refund_negative</code>	Test with a negative refund amount(amount must be positive)	<code>PaymentGateway</code>	
<code>test_refund_zero</code>	Test with a zero refund amount(amount must be positive)	<code>PaymentGateway</code>	
<code>test_refund_excess</code>	Test with a refund more than 15(refund cannot be more than 15)	<code>PaymentGateway</code>	
<code>test_payment_success</code>	Test with valid inputs	<code>PaymentGateway</code>	<code>refund_payment</code>

test_payment_fail	Test with PaymentGateway refund_payment unsuccessful	PaymentGateway	refund_payment
test_payment_exception	Test with PaymentGateway refund_payment throwing an Exception	PaymentGateway	refund_payment

Coverage Analysis

```

Command Prompt

C:\Users\Paul Wang\Documents\CISC 327\A3>pytest --cov=services --cov-report=html --cov-report=term tests/
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\Paul Wang\Documents\CISC 327\A3
plugins: cov-7.0.0, mock-3.15.1
collected 81 items

tests\test_add_book.py ..... [ 23%]
tests\test_book_catalog.py ... [ 27%]
tests\test_borrow_book.py ..... [ 41%]
tests\test_fees.py ..... [ 53%]
tests\test_pay_fees.py ..... [ 65%]
tests\test_refund.py ..... [ 74%]
tests\test_report.py ..... [ 80%]
tests\test_return.py ..... [ 91%]
tests\test_search.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.12.0-final-0

Name                Stmts  Miss  Cover
-----
services\__init__.py      0      0  100%
services\library_service.py 141      2   99%
services\payment_service.py  30     22   27%
-----
TOTAL                     171     24   86%
Coverage HTML written to dir htmlcov
===== 81 passed in 3.55s =====

```

Initially I was missing a few lines of coverage on unsuccessful attempts on PaymentGateway refund_payment along with the exception catching in both pay_late_fees and refund_late_fee_payment. I added test_pay_fee_exception, test_payment_fail and test_payment_exception. This allowed me to hit 99% coverage on library_service.py. The remaining 1% is two lines/branches of code which handle if a PaymentGateway was not provided. The functions are meant so that a PaymentGateway class is provided for testing. Therefore I don't see any reasons to cover these two lines of code since it is very unlikely to cause any issues.

```

# Use provided gateway or create new one
if payment_gateway is None:
    payment_gateway = PaymentGateway()

```

The 27% coverage on payment_service.py seems atrocious, however this testing was meant to cover all of library service. The code in payment_service also does not seem to be implemented yet.

Challenges and Solutions

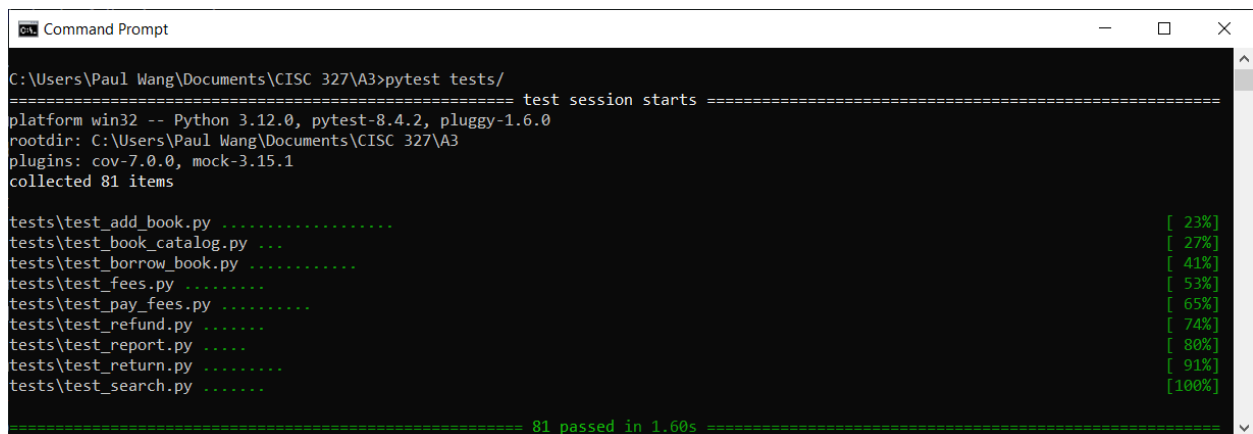
The first challenge I encountered is learning how to use mocks and stubs in pytest. The documentation is very long and I wasn't sure which functions were applicable to my use case. To solve this I asked AI(Queen's U Librechat) for an example and explanation of the code. It also told me that I needed to install `pytest-mock` and that it didn't come pre-installed with `pytest`.

The second challenge was getting my patch decorators to work. After some testing I gave up and asked AI which told me that decorators apply from the bottom up(why tho). This helped since it was mentioned literally nowhere in the `pytest-mock` documentation(it could just be a python thing I'm not familiar with).

The third challenge was converting every single test file previously written to use mocks and stubs. I needed to do this since the previous tests used database writing(I did this because I didn't know about mocks and stubs back then). This just took a lot of time and the solution was just to write code for a while.

Overall I see the use of mocks and stubs. It is very convenient to not worry about any pre-existing states or side effects of the testing. The ability of stubs and mocks to isolate a single function and test that function in exactly the way I want with minimal hassle is welcomed. The mocks are even more useful in allowing to check states halfway through the function.

Screenshots



```
Command Prompt
C:\Users\Paul Wang\Documents\CISC 327\A3>pytest tests/
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\Paul Wang\Documents\CISC 327\A3
plugins: cov-7.0.0, mock-3.15.1
collected 81 items

tests\test_add_book.py ..... [ 23%]
tests\test_book_catalog.py ... [ 27%]
tests\test_borrow_book.py ..... [ 41%]
tests\test_fees.py ..... [ 53%]
tests\test_pay_fees.py ..... [ 65%]
tests\test_refund.py ..... [ 74%]
tests\test_report.py ..... [ 80%]
tests\test_return.py ..... [ 91%]
tests\test_search.py ..... [100%]

===== 81 passed in 1.60s =====
```

```
Command Prompt
C:\Users\Paul Wang\Documents\CISC 327\A3>pytest --cov=services --cov-report=html --cov-report=term tests/
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\Paul Wang\Documents\CISC 327\A3
plugins: cov-7.0.0, mock-3.15.1
collected 81 items

tests\test_add_book.py ..... [ 23%]
tests\test_book_catalog.py ... [ 27%]
tests\test_borrow_book.py ..... [ 41%]
tests\test_fees.py ..... [ 53%]
tests\test_pay_fees.py ..... [ 65%]
tests\test_refund.py ..... [ 74%]
tests\test_report.py ..... [ 80%]
tests\test_return.py ..... [ 91%]
tests\test_search.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.12.0-final-0

Name                               Stmts  Miss  Cover
-----
services\__init__.py                 0      0   100%
services\library_service.py        141      2    99%
services\payment_service.py         30     22    27%
-----
TOTAL                               171     24    86%
Coverage HTML written to dir htmlcov
===== 81 passed in 3.55s =====
```

Coverage report: 86%

Files Functions Classes

coverage.py v7.11.2, created at 2025-11-09 19:00 -0500

File	function	statements	missing	excluded	coverage
C:\Users\Paul Wang\Documents\CISC 327\A3\services__init__.py	(no function)	0	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	add_book_to_catalog	19	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	borrow_book_by_patron	19	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	return_book_by_patron	13	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	calculate_late_fee_for_book	16	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	search_books_in_catalog	12	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	get_patron_status_report	15	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	pay_late_fees	20	1	0	95%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	refund_late_fee_payment	15	1	0	93%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\library_service.py	(no function)	12	0	0	100%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\payment_service.py	PaymentGateway.__init__	2	2	0	0%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\payment_service.py	PaymentGateway.process_payment	9	9	0	0%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\payment_service.py	PaymentGateway.refund_payment	7	7	0	0%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\payment_service.py	PaymentGateway.verify_payment_status	4	4	0	0%
C:\Users\Paul Wang\Documents\CISC 327\A3\services\payment_service.py	(no function)	8	0	0	100%
Total		171	24	0	86%